

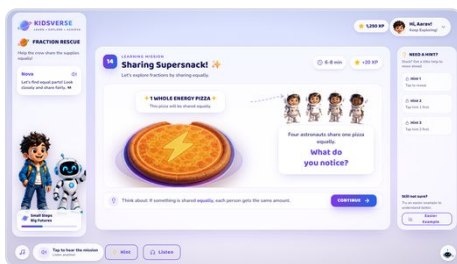
Content contract — the frontend side

Kidsverse+ — what the three lesson screens read, so the learning engine can produce it

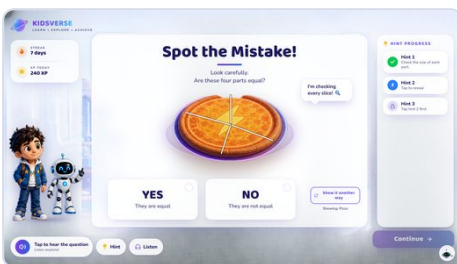
For backend / learning engine Date 8 September 2026 Status live in the app today

What changed, in one line

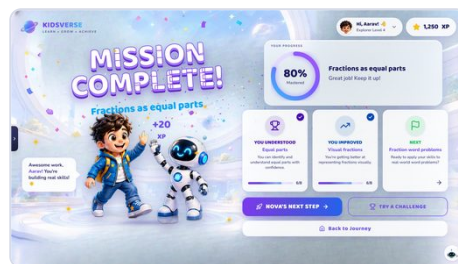
The three lesson screens no longer carry their words, hints, options or answers in code. Each renders one **learning package** — the JSON on the next pages. The Fractions mission ships as that JSON now, and the screens look exactly as they did. The engine replaces it by serving the same shape from an endpoint; nothing on the frontend changes.



14 · Discover — reads `mission` + `discover`



16 · Spot the Mistake — reads `check`



17 · Mission Complete — reads `complete` + `mission.xp`

1. Plugging in

```
VITE_CONTENT_API=https://api.example.com          # build / .env
GET {VITE_CONTENT_API}/learning-packages/{content_id}
→ 200, application/json, the package below, with content_id echoed back
```

Client behaviour (`src/content/index.js`):

- the bundled package renders on the first frame
- the remote package replaces it when it arrives and parses
- any failure — network, non-200, id mismatch — is logged and the bundled copy stays. A child never sees an empty screen because a service was slow.

So the endpoint can be built against the live app: point the env var at it and whatever it returns is what the child sees.

2. The package — complete, as it ships

Top-level metadata matches the “7 contents + 8 questions” package. The three screen blocks match the screens. `contents[]` and `questions[]` are arrays so the engine can return the full set; `selected` says which one the screen shows. Every string here is on screen today.

```
{
  "content_id": "fractions-equal-parts",
  "content_version": "1.0",
  "subject": "Mathematics",
  "grade": "4",
  "board": "CBSE",
  "topic": "Fractions",
  "subtopic": "Equal parts",
  "learning_objective": "Understand fractions as equal parts",
  "teaching_strategy": "visual",
  "source_asset_id": "pizza_4_parts",
  "mission": {
```

```

"code": "14",
"title": "Sharing Supersnack!",
"emoji": "🌟",
"subtitle": "Let's explore fractions by sharing equally.",
"duration_min": [
  6,
  8
],
"xp": 20,
"rail": {
  "title": "Fraction Rescue",
  "blurb": "Help the crew share the supplies equally!"
}
},
"discover": {
  "selected": 0,
  "contents": [
    {
      "content_no": 1,
      "type": "equal_sharing",
      "model": {
        "key": "pizza",
        "title": "1 Whole Energy Pizza",
        "caption": "This pizza will be shared equally."
      },
      "crew": {
        "count": 4,
        "image": "/art/new/astronauts.webp"
      },
      "prompt": {
        "statement": "Four astronauts share one pizza equally.",
        "question": "What do you notice?"
      },
      "think_about": {
        "text": "If something is shared equally, each person gets the same amount.",
        "emphasis": "equally"
      },
      "hints": [
        "Look at the pizza. How many equal parts could we make?",
        "If we cut it into 4 equal parts, what fraction does each astronaut get?",
        "Each astronaut gets one part out of four equal parts."
      ],
      "easier": {
        "enabled": true,
        "crew_count": 2,
        "statement": "Two astronauts share one pizza equally.",
        "label": "Sharing between 2"
      },
      "nova": {
        "speech": "Let's find equal parts! Look closely and share fairly. 🗣️",
        "voice": "Let's find equal parts. Look closely at the pizza and share it fairly."
      }
    }
  ]
},
"check": {
  "selected": 0,
  "questions": [
    {
      "question_id": "q_fraction_001",
      "type": "visual_yes_no",
      "title": "Spot the Mistake!",
      "instruction": "Look carefully.\nAre these four parts equal?",
      "split": [
        1,
        1,
        1,
        1
      ]
    }
  ]
}

```

```

],
"models": [
  {
    "key": "pizza",
    "label": "Pizza",
    "hints": [
      "Check the size of each part.",
      "Look at the crust lengths.",
      "Compare the toppings area."
    ],
    "feedback_wrong": "Hmm, look again at the crust. Is every slice the same size? 🕒"
  },
  {
    "key": "bar",
    "label": "Chocolate bar",
    "hints": [
      "Check the size of each chunk.",
      "Look along the top edge.",
      "Compare the widest chunk with the narrowest."
    ],
    "feedback_wrong": "Hmm, look again at the chunks. Is every chunk the same width? 🕒"
  },
  {
    "key": "line",
    "label": "Number line",
    "hints": [
      "Check the size of each jump.",
      "Look at the gaps between the marks.",
      "Compare the longest jump with the shortest."
    ],
    "feedback_wrong": "Hmm, look again at the jumps. Is every jump the same length? 🕒"
  }
],
"options": [
  {
    "key": "yes",
    "label": "YES",
    "sub": "They are equal"
  },
  {
    "key": "no",
    "label": "NO",
    "sub": "They are not equal"
  }
],
"answer": "no",
"feedback_correct": "Great observation! Not all four slices are the same. ✨",
"xp_on_correct": 10,
"nova": {
  "speech": "I'm checking every slice! 🕒",
  "voice": "Look carefully. Are these four parts equal?"
}
},
"complete": {
  "topic_label": "Fractions as equal parts",
  "encouragement": "Great job! Keep it up!",
  "nova": {
    "speech": "Awesome work, {name}! You're building real skills! 🌟"
  },
  "outcomes": [
    {
      "kind": "understood",
      "heading": "You understood",
      "skill": "Equal parts",
      "description": "You can identify and understand equal parts with confidence."
    }
  ]
},

```

```
{
  "kind": "improved",
  "heading": "You improved",
  "skill": "Visual fractions",
  "description": "You're getting better at representing fractions visually."
},
{
  "kind": "next",
  "heading": "Next",
  "skill": "Fraction word problems",
  "description": "Ready to apply your skills to real-world word problems?"
}
],
"next_step": "/journey"
}
```

3. Things the engine has to know

Topic	Rule
Pictures	For fraction questions the app draws the pizza, the chocolate bar and the number line itself from <code>split</code> . So <code>models[]</code> carries no image files for fractions — the three keys name the three renderers, and swapping the picture can never change the answer. For other subjects <code>models[n].image</code> is an uploaded PNG (sizes in the Content Slots PDF). Both land in the same slot.
Hints are per picture	“Look at the crust” is meaningless on a number line. Three hints per model, not three per question. The screen shows the hints of whichever picture is on screen.
<code>{name}</code>	In any text, replaced with the child’s first name on the client.
Options	Keys are free-form; <code>answer</code> is the key of the correct one. The check screen renders 2 today and the template accepts 2–4. Question types beyond yes/no (an MCQ over fractions, a scenario) need a second question template on the client — not yet built .
Selection	How <code>selected</code> is chosen — in sequence through the 7, or by strategy — is the open product decision. The shape supports either; the client currently shows index 0.
What not to send	Anything about the child: XP totals, mastery %, the progress bars on Mission Complete, streaks. The client computes those from the child’s record. <code>mission.xp</code> and <code>xp_on_correct</code> are the amounts <i>offered</i> , not balances.
Limits	Every text slot has a character limit and every image a size; they are in the Content Slots PDF. The screens are fixed-layout for a tablet, so an over-long title wraps onto the picture.

4. Where each field is read

Package field	Screen	Component
<code>mission.*</code> , <code>discover.*</code>	14 Discover	<code>src/screens/LessonDiscover.jsx</code>
<code>check.*</code>	16 Spot the Mistake	<code>src/screens/SpotMistake.jsx</code>
<code>complete.*</code> , <code>mission.xp</code>	17 Mission Complete	<code>src/screens/MissionComplete.jsx</code>
loader, remote fallback	—	<code>src/content/index.js</code>

Verified

All three screens render from the package with no visual change and no console errors; build passes. “Show it another way” switches to the package’s second model and its own hints. The remote path is wired and code-reviewed but has not been exercised against a live endpoint yet — that is the first thing to do together.